Extension TRIGO pour le compilateur Deca

Projet Génie Logiciel — Grenoble INP Ensimag

EL HABBOULI Hossam
EL HANAFY Aymen
EZ-ZAHORI Insaf
L'HICHOU Amr
REGRAGUI Ilyas

Équipe GL

22 janvier 2026

Table des matières

1 Introduction **4**
1.1 Contexte du projet Génie Logiciel . . . 4
1.2 Présentation du langage Deca . . . 4
1.3 Objectifs de l'extension TRIGO . . . 4
1.4 Contraintes imposées . . . 5
1.5 Organisation du document . . . 5

2 Rappels mathématiques **6**
2.1 Fonctions trigonométriques usuelles . . . 6
2.1.1 Définitions fondamentales . . . 6
2.1.2 Parité et symétries . . . 6
2.1.3 Relations fondamentales . . . 6
2.2 Séries de Taylor . . . 7
2.2.1 Définition générale . . . 7
2.2.2 Série de Taylor du cosinus . . . 7
2.2.3 Série de Taylor du sinus . . . 7
2.3 Réduction de domaine . . . 7
2.3.1 Motivation . . . 7
2.3.2 Réduction par périodicité . . . 7
2.3.3 Réduction par symétrie . . . 8
2.3.4 Réduction vers $[0, \pi/2]$ . . . 8
2.3.5 Réduction finale vers $[0, \pi/4]$ . . . 8
2.4 Fonctions réciproques . . . 8
2.4.1 Fonction arc sinus . . . 8
2.4.2 Fonction arc tangente . . . 8
2.5 Précision numérique et flottants . . . 8

3 Arithmétique flottante et notion d'ULP **9**
3.1 Représentation des nombres flottants . . . 9
3.2 Nombres normalisés et subnormaux . . . 9
3.3 Définition de l'ULP . . . 9
3.4 Expression de l'ULP . . . 10
3.5 Calcul pratique de l'exposant . . . 10
3.6 Utilisation de l'ULP dans le projet . . . 10

4 Architecture de l'extension TRIGO **11**
4.1 Choix d'implémentation . . . 11
4.2 Organisation de la classe Math . . . 11
4.3 Fonctions auxiliaires . . . 12
4.3.1 Valeur absolue . . . 12
4.3.2 Puissance entière . . . 12

4.3.3 Factorielle . . . 12
4.3.4 Racine carrée . . . 12
4.3.5 Calcul de l'exposant . . . 13
4.4 Constantes numériques . . . 13

5 Réduction de domaine **14**
5.1 Périodicité . . . 14
5.2 Symétries . . . 14
5.3 Réduction vers $[0, \pi]$ . . . 15
5.4 Réduction vers $[0, \pi/2]$ . . . 15
5.5 Réduction vers $[0, \pi/4]$ . . . 15
5.6 Impact sur la précision . . . 16

6 Implémentation de $\cos(x)$ **17**
6.1 Cas particuliers . . . 17
6.2 Réduction de domaine . . . 17
6.3 Approximation polynomiale . . . 18
6.4 Critère d'arrêt par ULP . . . 18
6.5 Version Java . . . 18
6.6 Version Deca . . . 19

7 Implémentation de $\sin(x)$ **20**
7.1 Domaine de définition . . . 20
7.2 Difficultés numériques . . . 20
7.3 Série de Taylor . . . 21
7.4 Réduction du domaine . . . 21
7.5 Critère d'arrêt par ULP . . . 22
7.6 Version Java . . . 22
7.7 Version Deca . . . 22

8 Implémentation de $\arcsin(x)$ **23**
8.1 Domaine de définition . . . 23
8.2 Difficultés numériques . . . 23
8.3 Série de Taylor . . . 24
8.4 Réduction du domaine . . . 24
8.5 Critère ULP . . . 24
8.6 Version Java . . . 25
8.7 Version Deca . . . 25

9 Implémentation de $\arctan(x)$ **26**
9.1 Propriétés . . . 26
9.2 Comportement pour grandes valeurs . . . 26
9.3 Réduction du domaine . . . 27
9.4 Série de Taylor . . . 27
9.5 Critère ULP . . . 27
9.6 Version Java . . . 28
9.7 Version Deca . . . 28

10 Optimisations numériques **29**
10.1 Limites des puissances et factorielles . . . 29
10.2 Méthode de Horner . . . 29
10.3 Horner pour $\cos(x)$ . . . 30
10.4 Horner pour $\sin(x)$ . . . 30

10.5 Gain en performances 30
10.6 Perspective FMA 31

11 Tests et validation **32**
11.1 Stratégie de tests 32
11.2 Tests Java 32
11.3 Valeurs remarquables 32
11.4 Grandes valeurs 33
11.5 Comparaison avec Java `Math` 33
11.6 Analyse des erreurs 33

12 Conclusion **34**
12.1 Bilan 34
12.2 Apports techniques 34
12.3 Difficultés rencontrées 34
12.4 Conclusion générale 35

Chapitre 1

Introduction

1.1 Contexte du projet Génie Logiciel

Le projet Génie Logiciel (GL) de deuxième année à Grenoble INP – Ensimag s'inscrit dans la continuité des enseignements fondamentaux en informatique et en ingénierie logicielle. Son objectif principal est la conception et l'implémentation d'un compilateur complet pour le langage Deca, un langage pédagogique conçu spécifiquement pour ce projet.

Ce projet couvre l'ensemble de la chaîne de compilation, depuis l'analyse lexicale et syntaxique jusqu'à la génération de code pour une machine abstraite. Il mobilise des compétences variées telles que la modélisation de langages, la manipulation d'arbres syntaxiques abstraits, la gestion des environnements de typage, ainsi que l'implémentation d'algorithmes non triviaux dans un cadre logiciel structuré.

Une particularité majeure du projet est son organisation en équipe, reproduisant un contexte proche du développement logiciel professionnel. Les étudiants doivent collaborer, partager le code via un gestionnaire de versions, définir des conventions communes et répartir les responsabilités. Le projet inclut également une part significative de travail d'analyse et de conception, notamment lors de l'implémentation d'extensions du langage, qui ne sont volontairement que partiellement spécifiées.

1.2 Présentation du langage Deca

Deca est un langage de programmation impératif et orienté objet, inspiré de langages existants tels que Java ou C, mais volontairement simplifié afin de permettre une implémentation complète dans le cadre d'un projet académique. Il supporte les types primitifs usuels (entiers, flottants, booléens), les structures de contrôle classiques, ainsi que les classes, méthodes et attributs.

Le langage est conçu pour être exécuté sur une machine abstraite, appelée IMA, qui définit un jeu d'instructions et un modèle d'exécution simplifié. Cette machine abstraite permet de se concentrer sur les aspects conceptuels de la compilation sans dépendre d'une architecture matérielle réelle.

Un aspect important de Deca est la présence d'une bibliothèque standard minimale, fournie sous forme de fichiers source Deca inclus via la directive include. Cette bibliothèque peut être enrichie par certaines extensions du langage, ce qui permet d'ajouter de nouvelles fonctionnalités sans modifier le cœur du compilateur. L'extension TRIGO s'inscrit précisément dans cette logique.

1.3 Objectifs de l'extension TRIGO

L'extension TRIGO a pour objectif d'enrichir la bibliothèque standard du langage Deca en y ajoutant des fonctions trigonométriques et mathématiques avancées. Plus précisément, elle

introduit une classe Math fournissant les méthodes suivantes :

cos(float)
sin(float)
arcsin(float)
arctan(float)
ulp(float)

Contrairement à d'autres parties du projet, l'extension TRIGO est spécifiée de manière volontairement peu guidée. Les signatures des méthodes sont imposées, mais leur implémentation relève entièrement du travail de l'équipe. Cela implique un effort important d'analyse mathématique, de choix algorithmiques et d'étude de la précision numérique.

L'objectif n'est pas uniquement d'obtenir des résultats fonctionnels, mais de proposer des implémentations robustes, cohérentes avec les contraintes du calcul en flottants simple précision (32 bits), et correctement documentées. L'extension constitue ainsi une part significative de la note finale et représente un travail comparable en complexité à certaines parties centrales du compilateur.

1.4 Contraintes imposées

L'implémentation de l'extension TRIGO est soumise à plusieurs contraintes fortes. Tout d'abord, les calculs doivent être effectués exclusivement en simple précision (float), conformément aux spécifications du langage Deca et de la machine abstraite IMA. L'utilisation de types en double précision est proscrite.

Ensuite, la bibliothèque standard est testée indépendamment du compilateur. Les implémentations ne doivent donc pas dépendre de comportements spécifiques du compilateur Deca, mais uniquement des règles du langage et de la machine abstraite. De plus, les tests fournis par l'équipe ne doivent pas exploiter des particularités internes de la bibliothèque développée, car ils seront exécutés avec une bibliothèque standard de référence.

Enfin, les fonctions mathématiques doivent respecter rigoureusement leurs domaines et codomaines mathématiques. Par exemple, la fonction arcsin doit refuser toute valeur en dehors de l'intervalle [1,1] [1,1], et retourner un résultat dans l'intervalle [/2,/2] [/2,/2]. Toute imprécision ou approximation doit être justifiée et analysée dans la documentation.

1.5 Organisation du document

Ce document est organisé de manière progressive afin de refléter la démarche suivie lors du développement de l'extension TRIGO. Après cette introduction, un chapitre est consacré aux rappels mathématiques nécessaires, incluant les séries de Taylor, les propriétés trigonométriques et les notions de précision numérique.

Les chapitres suivants détaillent les choix algorithmiques retenus pour chaque fonction, les techniques de réduction de domaine, ainsi que les stratégies d'approximation polynomiale. Une attention particulière est portée à l'analyse des erreurs numériques et à l'utilisation de l'ULP comme critère d'arrêt des séries.

Enfin, le document présente la validation expérimentale des implémentations à travers des tests réalisés en Java, ainsi qu'une discussion sur les optimisations possibles, notamment l'introduction de la méthode de Horner. L'objectif est de fournir une documentation complète, justifiant à la fois les choix théoriques et les décisions pratiques prises au cours du projet.

Chapitre 2

Rappels mathématiques

Ce chapitre présente les notions mathématiques nécessaires à la compréhension et à l'implémentation de l'extension `TRIGO`. Il s'appuie principalement sur l'analyse réelle, les séries de Taylor, les propriétés des fonctions trigonométriques ainsi que sur les bases de l'arithmétique en virgule flottante.

2.1 Fonctions trigonométriques usuelles

Les fonctions trigonométriques considérées dans ce projet sont :

$$\sin(x), \quad \cos(x), \quad \arcsin(x), \quad \arctan(x)$$

où x est un réel exprimé en radians.

2.1.1 Définitions fondamentales

Sur le cercle trigonométrique de rayon 1, pour un angle x mesuré en radians :
— $\cos(x)$ est l'abscisse du point correspondant,
— $\sin(x)$ est l'ordonnée du point correspondant.
Ces fonctions sont périodiques de période 2π :

$$\sin(x + 2\pi) = \sin(x), \qquad \cos(x + 2\pi) = \cos(x)$$

2.1.2 Parité et symétries

Les fonctions trigonométriques vérifient les propriétés suivantes :

$$\sin(-x) = -\sin(x) \quad \text{(fonction impaire)}$$
$$\cos(-x) = \cos(x) \quad \text{(fonction paire)}$$

Ces propriétés permettent de réduire le domaine de calcul aux réels positifs sans perte de généralité.

2.1.3 Relations fondamentales

Les relations trigonométriques suivantes sont largement exploitées :

$$\sin^2(x) + \cos^2(x) = 1$$
$$\sin\left(\frac{\pi}{2} - x\right) = \cos(x)$$
$$\cos\left(\frac{\pi}{2} - x\right) = \sin(x)$$

Ces identités permettent d'exprimer certaines fonctions en termes d'autres, facilitant ainsi leur implémentation.

2.2 Séries de Taylor

2.2.1 Définition générale

Soit f une fonction infiniment dérivable au voisinage de 0. La série de Taylor de f en 0 (série de Maclaurin) est définie par :

$$f(x) = \sum_{n=0}^{+\infty} \frac{f^{(n)}(0)}{n!} x^n$$

Lorsque la série converge vers $f(x)$, on obtient une approximation polynomiale de la fonction.

2.2.2 Série de Taylor du cosinus

La fonction $\cos(x)$ admet le développement suivant :

$$\cos(x) = \sum_{k=0}^{+\infty} (-1)^k \frac{x^{2k}}{(2k)!}$$

Les premiers termes sont :

$$\cos(x) \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!}$$

Cette série converge rapidement pour $|x| \leq \frac{\pi}{4}$, ce qui justifie une réduction préalable du domaine.

2.2.3 Série de Taylor du sinus

La fonction $\sin(x)$ admet le développement :

$$\sin(x) = \sum_{k=0}^{+\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

Les premiers termes sont :

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!}$$

Comme pour le cosinus, la convergence est optimale pour de petites valeurs de $|x|$.

2.3 Réduction de domaine

2.3.1 Motivation

Les séries de Taylor fournissent une bonne approximation uniquement sur un domaine restreint autour de 0. Afin de garantir précision et stabilité numérique, il est indispensable de ramener tout angle x dans un intervalle réduit.

2.3.2 Réduction par périodicité

Grâce à la périodicité des fonctions trigonométriques :

$$x \mapsto x \bmod 2\pi$$

permet de ramener tout angle dans l'intervalle $[0, 2\pi[$.

2.3.3 Réduction par symétrie

À l'aide des propriétés de parité :

$$\cos(x) = \cos(|x|)$$
$$\sin(x) = \operatorname{sgn}(x)\sin(|x|)$$

il est possible de réduire le calcul à $x \geq 0$.

2.3.4 Réduction vers $[0, \pi/2]$

Les identités suivantes sont utilisées :

$$\cos(x) = -\cos(\pi - x), \quad \sin(x) = \sin(\pi - x)$$

pour $x \in [\pi/2, \pi]$.

2.3.5 Réduction finale vers $[0, \pi/4]$

Lorsque $x > \pi/4$, on utilise la cofonction :

$$\sin(x) = \cos\left(\frac{\pi}{2} - x\right)$$

Cette dernière étape permet d'évaluer les séries de Taylor dans leur domaine de meilleure convergence.

2.4 Fonctions réciproques

2.4.1 Fonction arc sinus

La fonction $\arcsin(x)$ est définie sur $[-1, 1]$ et vérifie :

$$\arcsin : [-1, 1] \to \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$$

Son développement en série de Taylor est :

$$\arcsin(x) = \sum_{k=0}^{+\infty} \frac{(2k)!}{4^k (k!)^2 (2k+1)} x^{2k+1}$$

2.4.2 Fonction arc tangente

La fonction $\arctan(x)$ est définie sur $\mathbb{R}$ et vérifie :

$$\arctan(x) = \sum_{k=0}^{+\infty} (-1)^k \frac{x^{2k+1}}{2k+1} \quad \text{pour } |x| \leq 1$$

Pour $|x| > 1$, on utilise l'identité :

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right)$$

2.5 Précision numérique et flottants

Les calculs sont réalisés en simple précision (`float`), ce qui implique :
— une mantisse de 24 bits,
— un exposant compris entre -126 et 127.

Ces contraintes imposent un contrôle précis des erreurs numériques, justifiant l'introduction de la notion d'ULP, détaillée dans le chapitre suivant.

Chapitre 3

Arithmétique flottante et notion d'ULP

Ce chapitre présente les notions essentielles d'arithmétique en virgule flottante nécessaires à l'implémentation correcte et précise des fonctions trigonométriques de l'extension `TRIGO`. L'objectif n'est pas de détailler exhaustivement la norme IEEE 754, mais d'en extraire les éléments strictement utiles au projet.

3.1 Représentation des nombres flottants

Dans le cadre de ce projet, tous les calculs sont effectués en simple précision (`float`). Un nombre flottant est représenté sous la forme :

$$x = (-1)^s \times m \times 2^e$$

où :
- s est le bit de signe,
- m est la mantisse normalisée,
- e est l'exposant réel.

En simple précision :
- la mantisse possède 24 bits effectifs (23 bits stockés + 1 bit implicite),
- l'exposant réel est compris entre -126 et 127 pour les nombres normalisés.

3.2 Nombres normalisés et subnormaux

Un flottant est dit :
- **normalisé** si sa mantisse est de la forme $1.xxx_2$,
- **subnormal** s'il est trop petit pour être normalisé.

Le plus petit flottant strictement positif représentable est :

$$\texttt{MIN_VALUE} = 2^{-149} \approx 1.4 \times 10^{-45}$$

3.3 Définition de l'ULP

L'ULP (*Unit in the Last Place*) d'un flottant x est définie comme la distance entre x et le flottant immédiatement supérieur représentable :

$$\mathrm{ulp}(x) = |x^+ - x|$$

Intuitivement, l'ULP représente la plus petite variation significative que l'on puisse apporter à un flottant donné.

3.4 Expression de l'ULP

Pour un flottant normalisé de la forme :

$$x = m \times 2^e$$

l'ULP est donnée par :

$$\text{ulp}(x) = 2^{e-(p-1)}$$

où p est le nombre de bits significatifs (ici $p = 24$).

Ainsi, pour les `float` :

$$\text{ulp}(x) = 2^{e-23}$$

Pour $x = 0$, par convention :

$$\text{ulp}(0) = \texttt{MIN_VALUE}$$

3.5 Calcul pratique de l'exposant

Dans notre implémentation, l'exposant réel est déterminé sans manipulation binaire directe, en utilisant uniquement des opérations arithmétiques simples :

— divisions successives par 2 si $|x| \geq 2$,
— multiplications successives par 2 si $|x| < 1$.

Cette méthode, bien que non optimale en performance, est :

— simple à implémenter,
— portable,
— parfaitement adaptée au cadre pédagogique du projet.

3.6 Utilisation de l'ULP dans le projet

L'ULP est utilisée comme :

— critère d'arrêt dans les séries de Taylor,
— mesure de la précision atteinte,
— seuil de comparaison pour limiter les calculs inutiles.

Un terme de série est considéré comme négligeable dès que :

$$|\text{terme}| < \text{ulp}(\text{résultat courant})$$

Cette approche permet :

— d'adapter dynamiquement le nombre de termes,
— d'éviter le sur-calcul,
— de garantir une précision cohérente avec la représentation flottante.

Ce mécanisme est utilisé pour les fonctions `cos`, `sin`, `arcsin` et `arctan`.

Chapitre 4

Architecture de l'extension TRIGO

Ce chapitre décrit l'architecture logicielle de l'extension `TRIGO`. Il présente les choix structurants effectués pour l'implémentation de la bibliothèque mathématique, en particulier l'organisation de la classe `Math`, les fonctions auxiliaires développées, ainsi que les constantes numériques nécessaires au bon fonctionnement des algorithmes.

L'objectif principal de cette architecture est de garantir :
— la lisibilité du code,
— la modularité des calculs numériques,
— la réutilisabilité des fonctions internes,
— la conformité avec les contraintes du langage Deca.

4.1 Choix d'implémentation

L'extension `TRIGO` est implémentée exclusivement sous la forme d'une bibliothèque standard écrite en Deca, sans modification spécifique du compilateur en dehors de la gestion des directives `#include`.

Les choix d'implémentation majeurs sont les suivants :
— toutes les fonctions sont implémentées en simple précision (`float`),
— aucun accès direct à la représentation binaire IEEE 754 n'est utilisé,
— les algorithmes reposent uniquement sur des opérations arithmétiques élémentaires,
— les fonctions trigonométriques sont construites progressivement à partir de briques simples.

Ce choix garantit la portabilité du code et son adéquation avec l'esprit pédagogique du projet.

4.2 Organisation de la classe Math

La classe `Math` constitue le cœur de l'extension `TRIGO`. Elle regroupe l'ensemble des constantes, fonctions auxiliaires et fonctions mathématiques exposées à l'utilisateur.

L'organisation générale est la suivante :
— constantes numériques globales,
— fonctions auxiliaires internes (préfixées par `_`),
— fonctions trigonométriques publiques,
— fonctions inverses et utilitaires avancées.

Cette structuration permet une séparation claire entre :
— les outils internes de calcul,
— les fonctions visibles par l'utilisateur.

4.3 Fonctions auxiliaires

Les fonctions auxiliaires constituent la base de tous les calculs numériques de l'extension. Elles sont volontairement simples, explicites et indépendantes les unes des autres.

4.3.1 Valeur absolue

La fonction valeur absolue est définie par :

$$|x| = \begin{cases} -x & \text{si } x < 0 \\ x & \text{sinon} \end{cases}$$

Elle est utilisée dans :
— la gestion des symétries trigonométriques,
— les critères d'arrêt des séries,
— les comparaisons avec des seuils de précision.
Son implémentation est volontairement minimale afin d'éviter toute dépendance externe.

4.3.2 Puissance entière

La fonction de puissance entière calcule :

$$x^n = \underbrace{x \times x \times \cdots \times x}_{n \text{ fois}}$$

Elle est utilisée dans :
— les séries de Taylor,
— le calcul des termes polynomiaux,
— l'évaluation de certaines constantes intermédiaires.
L'algorithme retenu est un calcul itératif simple, suffisant compte tenu des faibles valeurs de n utilisées.

4.3.3 Factorielle

La factorielle d'un entier n est définie par :

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ \prod_{k=1}^{n} k & \text{sinon} \end{cases}$$

Cette fonction est essentielle pour :
— les coefficients des séries de Taylor,
— le calcul des développements de `sin`, `cos`, `arcsin` et `arctan`.
Le calcul est réalisé de manière itérative, avec des bornes volontairement faibles pour éviter les débordements.

4.3.4 Racine carrée

La racine carrée est approximée par la méthode de Newton–Raphson, qui consiste à résoudre :

$$y^2 = x$$

L'itération utilisée est :

$$y_{n+1} = \frac{1}{2}\left(y_n + \frac{x}{y_n}\right)$$

Cette méthode converge rapidement pour les valeurs positives et est utilisée notamment dans :
— la réduction de domaine de la fonction `arcsin`,
— le calcul intermédiaire de certaines transformations trigonométriques.

4.3.5 Calcul de l'exposant

Le calcul de l'exposant réel d'un flottant est réalisé sans manipulation binaire directe.
L'algorithme consiste à :
— diviser successivement par 2 si $|x| \geq 2$,
— multiplier successivement par 2 si $|x| < 1$.
Le nombre d'itérations permet de déterminer l'exposant réel e tel que :

$$1 \leq \frac{|x|}{2^e} < 2$$

Cette méthode est utilisée exclusivement pour le calcul de l'ULP.

4.4 Constantes numériques

Les constantes numériques définies dans la classe `Math` sont :
— π, utilisé pour toutes les fonctions trigonométriques,
— les bornes de l'exposant réel (`MIN_EXPONENT`, `MAX_EXPONENT`),
— le plus petit flottant strictement positif (`MIN_VALUE`),
— le plus grand flottant représentable (`MAX_VALUE`),
— le nombre de bits significatifs (`SIGNIFICAND_WIDTH`).
Ces constantes permettent de :
— garantir la cohérence numérique des calculs,
— implémenter correctement la fonction `ulp`,
— définir des critères d'arrêt adaptés à la précision des flottants.

Chapitre 5

Réduction de domaine

La réduction de domaine est une étape fondamentale dans l'implémentation des fonctions trigonométriques. Elle consiste à transformer une valeur d'entrée arbitraire en une valeur équivalente appartenant à un intervalle restreint sur lequel les approximations polynomiales sont à la fois plus précises et plus stables numériquement.

Sans réduction de domaine, les séries de Taylor convergeraient trop lentement, voire pas du tout, pour des valeurs éloignées de l'origine.

5.1 Périodicité

Les fonctions trigonométriques sont périodiques :

$$\cos(x + 2k\pi) = \cos(x), \quad \sin(x + 2k\pi) = \sin(x)$$

pour tout entier k.

Cette propriété permet de ramener toute valeur réelle x dans l'intervalle fondamental :

$$[0, 2\pi[$$

Dans l'implémentation, cette réduction est effectuée par :

$$x \leftarrow x - \left\lfloor \frac{x}{2\pi} \right\rfloor \cdot 2\pi$$

Ce calcul est volontairement réalisé à l'aide d'opérations arithmétiques simples afin de rester compatible avec les contraintes du langage Deca, qui ne fournit pas de fonction modulo flottant native.

La réduction par périodicité constitue la première étape de normalisation des angles.

5.2 Symétries

Les fonctions trigonométriques possèdent des propriétés de symétrie qui permettent de réduire davantage le domaine d'étude.

Pour le cosinus, on utilise la parité :

$$\cos(-x) = \cos(x)$$

Pour le sinus, on utilise l'imparité :

$$\sin(-x) = -\sin(x)$$

Ces propriétés permettent de :

— réduire l'intervalle d'étude aux valeurs positives,
— isoler la gestion du signe dans une variable dédiée,
— simplifier les calculs polynomiaux.

Dans l'implémentation, le signe est stocké séparément, et l'angle est rendu positif avant toute autre transformation.

5.3 Réduction vers $[0, \pi]$

Après la réduction par périodicité et symétrie, l'angle appartient à l'intervalle $[0, 2\pi[$.

On exploite alors l'identité :

$$\cos(x) = \cos(2\pi - x)$$

Cette relation permet de ramener tout angle dans l'intervalle :

$$[0, \pi]$$

Concrètement, si $x > \pi$, on applique :

$$x \leftarrow 2\pi - x$$

Cette étape est essentielle pour limiter la croissance des puissances dans les séries de Taylor, et donc réduire les erreurs d'arrondi.

5.4 Réduction vers $[0, \pi/2]$

Le cosinus vérifie également :

$$\cos(x) = -\cos(\pi - x)$$

Cette identité permet de réduire encore le domaine à :

$$[0, \pi/2]$$

Lorsque $x > \pi/2$, on applique :

$$x \leftarrow \pi - x$$

et on inverse le signe du résultat final.

Cette étape permet de :
— conserver une cohérence du signe,
— réduire la valeur maximale de x utilisée dans les polynômes,
— améliorer significativement la précision numérique.

5.5 Réduction vers $[0, \pi/4]$

Les polynômes de Taylor convergent d'autant plus vite que $|x|$ est petit.

Pour atteindre une précision satisfaisante avec un nombre raisonnable de termes, on réduit l'angle final dans l'intervalle :

$$[0, \pi/4]$$

Cette réduction repose sur la cofonctionnalité :

$$\sin(x) = \cos\left(\frac{\pi}{2} - x\right)$$

Ainsi, pour le sinus :

$$\text{si } x > \frac{\pi}{4}, \quad \sin(x) = \cos\left(\frac{\pi}{2} - x\right)$$

Cette transformation permet :

— d'utiliser un seul type de polynôme dans certaines plages,
— de garantir une meilleure convergence,
— de limiter le nombre de termes nécessaires.

Dans le cas du cosinus, la réduction finale s'arrête à $[0, \pi/2]$, le polynôme étant déjà suffisamment stable dans cet intervalle après les réductions précédentes.

5.6 Impact sur la précision

La réduction de domaine a un impact direct sur la précision numérique des fonctions trigonométriques.

Sans réduction, les termes de la série de Taylor croissent rapidement :

$$x^n \to \infty \quad \text{lorsque } |x| \gg 1$$

Avec réduction, on garantit :

$$|x| \leq \frac{\pi}{4}$$

Ce choix permet :
— une convergence rapide des séries,
— une diminution des erreurs d'arrondi,
— un critère d'arrêt basé sur l'ULP fiable et efficace.

La réduction de domaine est donc un élément central de l'architecture numérique de l'extension `TRIGO`, conditionnant à la fois la performance et la robustesse des calculs.

Chapitre 6

Implémentation de $\cos(x)$

Ce chapitre décrit en détail l'implémentation de la fonction cosinus dans l'extension TRIGO. L'objectif est de fournir une approximation précise, robuste et compatible avec les contraintes du langage Deca, en s'appuyant uniquement sur des opérations élémentaires.

L'implémentation repose sur quatre piliers :

— la gestion explicite des cas particuliers,
— une réduction progressive du domaine,
— une approximation polynomiale basée sur les séries de Taylor,
— un critère d'arrêt fondé sur l'ULP.

6.1 Cas particuliers

Avant toute approximation numérique, certains cas doivent être traités explicitement. Ces valeurs correspondent à des angles remarquables pour lesquels le cosinus est connu exactement.

On considère un seuil de tolérance ε afin de tenir compte des erreurs de représentation en virgule flottante.

Les cas particuliers traités sont :

$$\begin{aligned} \cos(0) &= 1 \\ \cos\left(\frac{\pi}{2}\right) &= 0 \\ \cos(\pi) &= -1 \\ \cos\left(\frac{3\pi}{2}\right) &= 0 \\ \cos(2\pi) &= 1 \end{aligned}$$

Formellement, pour une valeur x normalisée :

$$\text{si } |x - x_0| < \varepsilon \Rightarrow \cos(x) = \cos(x_0)$$

Cette étape permet :

— d'éviter des erreurs numériques inutiles,
— d'améliorer la stabilité globale,
— de garantir des résultats exacts pour les valeurs critiques.

6.2 Réduction de domaine

Après traitement des cas particuliers, l'argument est réduit progressivement afin d'amener le calcul dans une zone où l'approximation polynomiale est fiable.

Les étapes sont les suivantes :

1. réduction par périodicité vers $[0, 2\pi[$,
2. exploitation de la parité : $\cos(-x) = \cos(x)$,
3. réduction vers $[0, \pi]$ via $\cos(x) = \cos(2\pi - x)$,
4. réduction vers $[0, \pi/2]$ via $\cos(x) = -\cos(\pi - x)$.

À chaque étape, un facteur de signe est mis à jour afin de préserver la valeur correcte du cosinus.

Cette réduction garantit que l'argument final satisfait :

$$0 \leq x \leq \frac{\pi}{2}$$

6.3 Approximation polynomiale

L'approximation du cosinus est basée sur son développement de Taylor autour de zéro :

$$\cos(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k}}{(2k)!}$$

Dans l'implémentation, ce développement est évalué terme par terme :

$$\cos(x) \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

Cette approche présente plusieurs avantages :
— elle est simple à implémenter,
— elle ne nécessite aucune fonction avancée,
— elle permet un contrôle fin de la précision.

Les puissances et les factorielles sont calculées à l'aide de fonctions auxiliaires explicites, afin de rester compatibles avec Deca.

6.4 Critère d'arrêt par ULP

Contrairement à une approximation à ordre fixe, l'implémentation utilise un critère d'arrêt adaptatif fondé sur l'ULP (*Unit in the Last Place*).

À chaque itération, on compare la valeur absolue du terme courant T_k avec l'ULP du résultat partiel :

$$|T_k| < \text{ulp}(\text{résultat})$$

Dès que cette condition est satisfaite, l'ajout du terme n'a plus d'impact significatif sur la valeur finale du flottant, et la boucle est interrompue.

Ce choix permet :
— d'adapter automatiquement le nombre de termes,
— d'éviter des calculs inutiles,
— de garantir une précision cohérente avec la représentation IEEE 754.

6.5 Version Java

La version Java est utilisée pour le prototypage et la validation numérique. Elle reproduit fidèlement les choix d'implémentation de la version Deca.

Les caractéristiques principales sont :
— utilisation exclusive du type `float`,
— implémentation explicite des réductions de domaine,

— critère d'arrêt basé sur l'ULP,
— absence de toute fonction mathématique native de Java.

Les tests Java permettent de comparer les résultats obtenus avec ceux de `java.lang.Math.cos`, et de mesurer l'erreur relative sur un large ensemble de valeurs.

6.6 Version Deca

La version Deca reprend exactement la même structure algorithmique que la version Java.

Les adaptations principales concernent :
— l'absence de tableaux dynamiques,
— la gestion manuelle du signe,
— l'utilisation exclusive des primitives du langage.

Cette cohérence entre les deux versions garantit que les résultats observés lors des tests Java sont représentatifs du comportement réel du compilateur Deca une fois l'extension intégrée.

L'implémentation finale de `cos(x)` constitue ainsi une base solide pour les autres fonctions trigonométriques de l'extension TRIGO.

Chapitre 7

Implémentation de $\sin(x)$

La fonction sinus est l'une des fonctions trigonométriques fondamentales. Bien que sa définition mathématique soit simple, son implémentation numérique nécessite une attention particulière afin de garantir une bonne précision sur l'ensemble des valeurs d'entrée possibles.

Ce chapitre présente l'implémentation de la fonction $\sin(x)$ dans l'extension TRIGO, en détaillant les difficultés numériques rencontrées, les méthodes de réduction de domaine utilisées, ainsi que l'approximation polynomiale retenue.

7.1 Domaine de définition

La fonction sinus est définie sur l'ensemble des réels :

$$\sin : \mathbb{R} \to [-1, 1]$$

Elle est :
— **périodique** de période 2π,
— **impaire** : $\sin(-x) = -\sin(x)$,
— continue et dérivable sur $\mathbb{R}$.

Ces propriétés sont exploitées directement dans l'implémentation afin de réduire le domaine de calcul et améliorer la stabilité numérique.

Les valeurs remarquables suivantes sont traitées explicitement :

$$\begin{aligned} \sin(0) &= 0 \\ \sin\left(\frac{\pi}{2}\right) &= 1 \\ \sin(\pi) &= 0 \\ \sin\left(\frac{3\pi}{2}\right) &= -1 \\ \sin(2\pi) &= 0 \end{aligned}$$

7.2 Difficultés numériques

Le calcul numérique de $\sin(x)$ pose plusieurs problèmes :
— la série de Taylor converge lentement lorsque $|x|$ est grand,
— les puissances élevées de x amplifient les erreurs d'arrondi,
— la périodicité impose une réduction préalable de l'argument,
— l'évaluation naïve est coûteuse en opérations.

Sans réduction de domaine, l'approximation polynomiale devient rapidement inexploitable. Une implémentation robuste doit donc impérativement ramener l'argument dans une zone où la série converge rapidement.

7.3 Série de Taylor

Le développement en série de Taylor de $\sin(x)$ autour de zéro est donné par :

$$\sin(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{(2k+1)!}$$

Les premiers termes sont :

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots$$

Cette série est particulièrement efficace lorsque $|x|$ est petit, en pratique pour :

$$|x| \leq \frac{\pi}{4}$$

C'est dans cet intervalle que l'approximation polynomiale est utilisée dans l'implémentation finale.

7.4 Réduction du domaine

Afin de ramener l'argument dans un domaine favorable, plusieurs propriétés mathématiques sont exploitées.

Périodicité

La périodicité de la fonction sinus permet d'écrire :

$$\sin(x) = \sin(x \bmod 2\pi)$$

L'argument est donc d'abord normalisé dans l'intervalle $[0, 2\pi[$.

Parité

La fonction sinus étant impaire :

$$\sin(-x) = -\sin(x)$$

On se ramène ainsi au cas $x \geq 0$ en conservant le signe final.

Symétries

On utilise ensuite les identités suivantes :

$$\sin(\pi - x) = \sin(x)$$
$$\sin(\pi + x) = -\sin(x)$$

Ces relations permettent de ramener l'argument dans l'intervalle $[0, \pi/2]$.

Cofonction

Enfin, pour $x \in]\pi/4, \pi/2]$, on utilise la relation de cofonction :

$$\sin(x) = \cos\left(\frac{\pi}{2} - x\right)$$

Cela permet de réduire le calcul final à l'intervalle :

$$x \in [0, \pi/4]$$

7.5 Critère d'arrêt par ULP

Plutôt que de fixer arbitrairement le nombre de termes de la série, l'implémentation utilise un critère d'arrêt basé sur l'ULP.

À chaque itération, le terme courant T_k est comparé à l'ULP du résultat partiel :

$$|T_k| < \text{ulp}(\text{résultat})$$

Lorsque cette condition est vérifiée, l'ajout du terme n'affecte plus la valeur du flottant représenté et la boucle est arrêtée.

Ce critère garantit :

— une précision adaptée au format `float`,
— une convergence automatique,
— un compromis optimal entre coût et précision.

7.6 Version Java

La version Java de $\sin(x)$ sert de référence expérimentale. Elle implémente exactement les mêmes étapes que la version Deca :

— normalisation de l'argument,
— réduction de domaine par symétries,
— approximation polynomiale adaptative,
— critère d'arrêt basé sur l'ULP.

Les résultats obtenus sont comparés à ceux de `Math.sin` afin d'évaluer l'erreur absolue et relative sur un large ensemble de valeurs.

7.7 Version Deca

La version Deca est directement dérivée de la version Java, en respectant les contraintes du langage :

— absence de bibliothèques mathématiques avancées,
— gestion explicite des flottants,
— utilisation exclusive de fonctions auxiliaires définies dans `Math.decah`.

Cette implémentation garantit une cohérence complète avec l'extension TRIGO et permet une intégration progressive dans le compilateur Deca.

Chapitre 8

Implémentation de $\arcsin(x)$

La fonction arc sinus est plus délicate à implémenter que les fonctions trigonométriques directes. Contrairement à $\sin(x)$ ou $\cos(x)$, la fonction $\arcsin(x)$ n'est définie que sur un domaine restreint et présente des problèmes de convergence numérique aux bornes de ce domaine.

Ce chapitre décrit les choix mathématiques et algorithmiques effectués pour implémenter $x)$ dans l'extension TRIGO, en respectant les contraintes du langage Deca.

8.1 Domaine de définition

La fonction arc sinus est définie uniquement sur l'intervalle :

$$x \in [-1, 1]$$

Son image est donnée par :

$$\arcsin(x) \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right]$$

Toute valeur en dehors de cet intervalle n'a pas de sens mathématique réel. Dans notre implémentation, un appel à $\arcsin(x)$ avec $|x| > 1$ provoque donc une erreur explicite, ce qui correspond au comportement attendu dans la spécification de l'extension TRIGO.

Les cas particuliers suivants sont traités explicitement :

$$\begin{aligned} \arcsin(0) &= 0 \\ \arcsin(1) &= \frac{\pi}{2} \\ \arcsin(-1) &= -\frac{\pi}{2} \end{aligned}$$

Ces valeurs sont détectées à l'aide d'un seuil ε afin de tenir compte des erreurs de représentation en virgule flottante.

8.2 Difficultés numériques

L'implémentation de $\arcsin(x)$ pose plusieurs difficultés numériques majeures :

— la convergence lente de la série de Taylor lorsque $|x|$ est proche de 1,
— la perte de précision due aux puissances élevées de x,
— l'amplification des erreurs d'arrondi près des bornes du domaine,
— l'absence de primitives mathématiques avancées dans Deca.

En particulier, la série de Taylor classique de $\arcsin(x)$ converge très mal lorsque $x \to \pm 1$. Une implémentation naïve serait donc inutilisable sur une large partie du domaine.

8.3 Série de Taylor

Le développement en série de Taylor de $\arcsin(x)$ autour de zéro est donné par :

$$\arcsin(x) = \sum_{k=0}^{\infty} \frac{(2k)!}{4^k (k!)^2 (2k+1)} x^{2k+1}$$

Les premiers termes sont :

$$\arcsin(x) \approx x + \frac{x^3}{6} + \frac{3x^5}{40} + \frac{5x^7}{112} + \cdots$$

Cette série converge rapidement pour les petites valeurs de $|x|$, typiquement pour $|x| \leq 0.5$. Dans cette zone, elle constitue une approximation efficace et simple à implémenter.

Chaque terme est calculé explicitement à l'aide de fonctions auxiliaires pour les puissances et les factorielles, afin de rester compatible avec Deca.

8.4 Réduction du domaine

Pour améliorer la convergence lorsque $|x|$ est proche de 1, on utilise une identité trigonométrique classique :

$$\arcsin(x) = \frac{\pi}{2} - \arcsin\left(\sqrt{1-x^2}\right), \quad x \geq 0$$

Cette transformation permet de ramener le calcul dans une zone où l'argument de la fonction arcsin est petit, ce qui accélère considérablement la convergence de la série de Taylor.

La fonction $\arcsin(x)$ étant impaire, on exploite également la propriété :

$$\arcsin(-x) = -\arcsin(x)$$

Ainsi, l'algorithme suit les étapes suivantes :

1. gestion du signe,
2. traitement des cas particuliers,
3. réduction du domaine si $x > 0.5$,
4. évaluation de la série de Taylor.

La racine carrée est approximée par la méthode de Newton–Raphson, déjà implémentée comme fonction auxiliaire.

8.5 Critère ULP

Plutôt que de fixer arbitrairement le nombre de termes de la série, l'implémentation utilise un critère d'arrêt basé sur l'ULP.

À chaque itération, on compare la valeur absolue du terme courant T_k avec l'ULP du résultat partiel :

$$|T_k| < \text{ulp}(\text{résultat})$$

Dès que cette condition est satisfaite, l'ajout du terme n'a plus d'effet sur la valeur finale du flottant et la boucle est interrompue.

Ce critère garantit :

— une précision adaptée à la représentation IEEE 754,
— une convergence automatique,
— une robustesse numérique accrue.

8.6 Version Java

La version Java de $\arcsin(x)$ est utilisée pour le prototypage et les tests. Elle implémente strictement les mêmes algorithmes que la version Deca :

— réduction de domaine,
— série de Taylor adaptative,
— critère d'arrêt par ULP,
— calcul explicite des fonctions auxiliaires.

Les résultats sont comparés à ceux de `Math.arcsin` de Java afin d'évaluer l'erreur absolue et relative sur l'ensemble du domaine $[-1, 1]$.

8.7 Version Deca

La version Deca reprend fidèlement la structure de la version Java, en tenant compte des contraintes spécifiques du langage :

— absence de méthodes statiques,
— gestion explicite des erreurs,
— utilisation exclusive des types primitifs,
— compatibilité avec l'extension TRIGO.

Cette cohérence entre les deux implémentations permet de valider le comportement de $\arcsin(x)$ avant l'intégration complète dans le compilateur Deca.

Chapitre 9

Implémentation de $\arctan(x)$

La fonction arc tangente, notée $\arctan(x)$, est l'inverse partielle de la fonction tangente. Contrairement à sin et cos, elle n'est pas périodique et présente un comportement asymptotique qui impose un traitement numérique spécifique.

Ce chapitre décrit l'implémentation numérique de $\arctan(x)$ dans l'extension TRIGO, en mettant l'accent sur la réduction de domaine, la convergence de la série de Taylor et le contrôle de la précision par l'ULP.

9.1 Propriétés

La fonction $\arctan(x)$ est définie sur $\mathbb{R}$ et vérifie :

$$\arctan : \mathbb{R} \to \left]-\frac{\pi}{2}, \frac{\pi}{2}\right[$$

Ses principales propriétés sont :

— fonction **impaire** :

$$\arctan(-x) = -\arctan(x)$$

— fonction **strictement croissante**,

— limites asymptotiques :

$$\lim_{x\to+\infty} \arctan(x) = \frac{\pi}{2}, \quad \lim_{x\to-\infty} \arctan(x) = -\frac{\pi}{2}$$

Ces propriétés sont exploitées directement pour réduire le domaine de calcul et éviter les divergences numériques.

9.2 Comportement pour grandes valeurs

Le développement en série de Taylor de $\arctan(x)$ autour de zéro ne converge que pour :

$$|x| \leq 1$$

En revanche, pour les grandes valeurs de $|x|$, la fonction se rapproche asymptotiquement de $\pm\frac{\pi}{2}$. Afin de traiter correctement ces cas, on utilise l'identité suivante :

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right) \quad \text{pour } x > 0$$

et, par impaireté :

$$\arctan(x) = -\frac{\pi}{2} + \arctan\left(\frac{1}{|x|}\right) \quad \text{pour } x < 0$$

Cette transformation permet de ramener tout calcul vers un argument strictement inférieur à 1 en valeur absolue.

9.3 Réduction du domaine

L'implémentation suit les étapes suivantes :

1. gestion du signe par impaireté :

$$\arctan(-x) = -\arctan(x)$$

2. test des valeurs proches de zéro,
3. réduction pour $|x| > 1$ via la relation :

$$\arctan(x) = \frac{\pi}{2} - \arctan\left(\frac{1}{x}\right)$$

À l'issue de cette réduction, le calcul est toujours effectué sur un argument appartenant à l'intervalle :

$$x \in [0, 1]$$

ce qui garantit une bonne convergence de la série de Taylor.

9.4 Série de Taylor

Le développement en série de Taylor de $\arctan(x)$ autour de zéro est :

$$\arctan(x) = \sum_{k=0}^{\infty} (-1)^k \frac{x^{2k+1}}{2k+1}$$

Les premiers termes sont :

$$\arctan(x) \approx x - \frac{x^3}{3} + \frac{x^5}{5} - \frac{x^7}{7} + \cdots$$

Cette série converge relativement lentement lorsque x est proche de 1, ce qui justifie l'usage d'un critère d'arrêt adaptatif basé sur l'ULP plutôt qu'un ordre fixe.

9.5 Critère ULP

À chaque itération, le terme courant T_k est évalué et comparé à l'ULP du résultat partiel :

$$|T_k| < \text{ulp}(\text{résultat})$$

Lorsque cette condition est satisfaite, le terme ajouté n'a plus d'impact sur la valeur du flottant représenté.

Ce critère permet :

— d'éviter des itérations inutiles,

— d'adapter automatiquement la précision au format `float`,

— d'obtenir un résultat stable même pour $x \approx 1$.

Un seuil de sécurité sur le nombre maximal d'itérations est néanmoins conservé afin d'éviter toute boucle infinie.

9.6 Version Java

La version Java de $\arctan(x)$ sert de banc d'essai numérique. Elle implémente :

— la réduction du domaine par impaireté,

— la transformation pour $|x| > 1$,

— la série de Taylor avec critère ULP.

Les résultats sont comparés à ceux de `Math.arctan` afin d'évaluer la précision obtenue sur l'ensemble du domaine réel.

Les écarts observés restent de l'ordre de quelques ULP, ce qui est cohérent avec les objectifs de l'extension TRIGO.

9.7 Version Deca

La version Deca reprend exactement la même structure algorithmique que la version Java, en utilisant uniquement les fonctions auxiliaires définies dans la bibliothèque standard :

— calcul des puissances entières,

— calcul de l'ULP,

— gestion explicite des flottants.

Cette implémentation respecte les contraintes du langage Deca et garantit une cohérence complète entre les tests Java et l'exécution finale dans le compilateur.

Chapitre 10

Optimisations numériques

L'implémentation initiale des fonctions trigonométriques repose sur des développements en série de Taylor utilisant explicitement des puissances et des factorielles. Bien que correcte sur le plan mathématique, cette approche présente des limitations importantes en termes de coût algorithmique, de stabilité numérique et de performance.

Ce chapitre présente les optimisations envisagées et mises en œuvre, en particulier l'utilisation de la méthode de Horner, ainsi que les perspectives offertes par l'instruction FMA.

10.1 Limites des puissances et factorielles

Dans les premières versions, les séries de Taylor sont évaluées sous la forme :

$$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

Chaque terme nécessite :
— un calcul de puissance x^n,
— un calcul de factorielle $n!$,
— une division flottante.

Cette approche présente plusieurs inconvénients :
— **coût élevé** : les fonctions `_pow` et `_fact` sont appelées à chaque itération,
— **accumulation d'erreurs** : chaque opération flottante introduit un arrondi,
— **mauvaise localité numérique** : les puissances élevées deviennent rapidement très petites ou très grandes.

Ces limitations motivent l'introduction d'une méthode plus stable et plus efficace pour l'évaluation des polynômes.

10.2 Méthode de Horner

La méthode de Horner permet d'évaluer un polynôme sous une forme factorisée, réduisant drastiquement le nombre d'opérations nécessaires.

Un polynôme classique :

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n$$

peut être réécrit sous la forme de Horner :

$$P(x) = a_0 + x(a_1 + x(a_2 + \cdots + x a_n))$$

Les avantages principaux sont :
— uniquement des multiplications et additions,

— réduction du nombre total d'opérations,
— meilleure stabilité numérique,
— structure parfaitement adaptée à une implémentation itérative.

Dans le contexte du calcul flottant, Horner limite également l'accumulation d'erreurs d'arrondi.

10.3 Horner pour $\cos(x)$

Le développement de Taylor du cosinus est :

$$\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots$$

En posant $y = x^2$, ce polynôme peut être réécrit sous forme de Horner :

$$\cos(x) \approx 1 + y\left(-\frac{1}{2!} + y\left(\frac{1}{4!} + y\left(-\frac{1}{6!} + \cdots\right)\right)\right)$$

Cette réécriture permet :
— de supprimer les appels à `_pow` et `_fact`,
— de limiter le calcul à une suite de multiplications par y,
— d'intégrer facilement un critère d'arrêt basé sur l'ULP.

L'évaluation se fait alors de manière descendante, ce qui améliore la stabilité lorsque $|x|$ est petit, cas dans lequel la réduction de domaine a ramené l'argument.

10.4 Horner pour $\sin(x)$

De manière similaire, la série de Taylor du sinus :

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

peut être factorisée en posant $y = x^2$:

$$\sin(x) = x\left(1 + y\left(-\frac{1}{3!} + y\left(\frac{1}{5!} + y\left(-\frac{1}{7!} + \cdots\right)\right)\right)\right)$$

Cette forme présente plusieurs avantages :
— le facteur x est appliqué une seule fois,
— la structure polynomiale est similaire à celle du cosinus,
— les erreurs d'arrondi sont mieux contrôlées.

L'utilisation de Horner rend également la transition vers une implémentation optimisée par FMA particulièrement naturelle.

10.5 Gain en performances

Le passage de l'évaluation naïve à la méthode de Horner entraîne :
— une réduction significative du nombre de multiplications,
— la suppression des divisions par des factorielles,
— une diminution du nombre total d'itérations nécessaires.

Sur les tests Java effectués, on observe :
— une accélération mesurable du temps d'exécution,
— une précision équivalente, voire légèrement améliorée,
— une meilleure robustesse pour les arguments proches des bornes du domaine réduit.

Ces gains sont particulièrement visibles lorsque les fonctions sont appelées de manière répétée.

10.6 Perspective FMA

L'instruction FMA (*Fused Multiply-Add*) permet de calculer :

$$a + b \times c$$

en une seule opération, avec un unique arrondi.

Dans le contexte de Horner, chaque étape est précisément de la forme :

$$r \leftarrow a + x \times r$$

L'utilisation de FMA permettrait :

— de réduire le nombre d'erreurs d'arrondi,

— d'améliorer encore la précision finale,

— d'optimiser les performances sur les architectures modernes.

Bien que l'instruction FMA ne soit pas explicitement disponible dans le langage Deca, la génération de code assembleur pourrait, à terme, exploiter cette instruction pour les expressions de type `a + b * c`.

Cette optimisation constitue une perspective naturelle pour une évolution future de l'extension TRIGO.

Chapitre 11

Tests et validation

La mise en œuvre de fonctions numériques approchées impose une phase de tests rigoureuse. L'objectif n'est pas uniquement de vérifier la correction fonctionnelle, mais aussi d'évaluer la précision numérique, la stabilité et le comportement aux limites du domaine de définition.

Ce chapitre décrit la stratégie de tests adoptée, les résultats obtenus en Java, ainsi que l'analyse des écarts observés par rapport aux fonctions de la bibliothèque standard `Math` de Java.

11.1 Stratégie de tests

La stratégie de tests repose sur plusieurs principes fondamentaux :

— tester d'abord en Java, avant l'intégration dans le compilateur Deca,
— rester strictement dans le type `float`, conformément aux contraintes du projet,
— comparer systématiquement avec les fonctions de référence `Math.cos`, `Math.sin`, `Math.asin` et `Math.atan`,
— couvrir l'ensemble des cas remarquables et des zones numériques sensibles.

Les tests ont été conçus pour être simples, lisibles et reproductibles, afin de faciliter l'interprétation des résultats.

11.2 Tests Java

Les fonctions ont été testées à l'aide de programmes Java autonomes, placés dans un répertoire séparé du projet principal. Cette approche permet :

— une compilation et une exécution rapides,
— une isolation complète du code de test,
— une comparaison directe avec les fonctions natives de Java.

Chaque test affiche :

— la valeur d'entrée,
— le résultat obtenu par l'implémentation TRIGO,
— le résultat de la bibliothèque `Math`,
— l'erreur absolue et relative.

Cette présentation facilite l'analyse qualitative et quantitative des écarts numériques.

11.3 Valeurs remarquables

Un soin particulier a été apporté aux valeurs remarquables, pour lesquelles les résultats sont connus analytiquement :

— $\cos(0) = 1$, $\sin(0) = 0$,
— $\cos(\pi/2) = 0$, $\sin(\pi/2) = 1$,

— $\cos(\pi) = -1$, $\sin(\pi) = 0$,
— $(0) = 0$, $(0) = 0$.

Ces valeurs servent de tests de non-régression et permettent de valider la gestion des cas particuliers implémentés explicitement dans le code.

Les résultats obtenus sont conformes aux valeurs théoriques, à une tolérance numérique proche de l'ULP.

11.4 Grandes valeurs

Les fonctions trigonométriques étant périodiques, il est essentiel de tester leur comportement pour des arguments de grande magnitude.

Des tests ont été réalisés pour :

$$x \in \{10^2, 10^4, 10^6, 10^8\}$$

Ces tests permettent de valider :
— la réduction de domaine,
— l'absence de débordement,
— la stabilité des résultats après normalisation.

Les résultats montrent que la réduction modulo 2π permet de maintenir une précision correcte, même pour des arguments très grands, dans les limites de la précision du type `float`.

11.5 Comparaison avec Java `Math`

La comparaison avec les fonctions de la bibliothèque standard Java constitue une référence fiable, celles-ci étant hautement optimisées et conformes à la norme IEEE 754.

Pour chaque fonction, l'erreur absolue est définie par :

$$\varepsilon_{\text{abs}} = |f_{\text{TRIGO}}(x) - f_{\text{Java}}(x)|$$

et l'erreur relative par :

$$\varepsilon_{\text{rel}} = \frac{|f_{\text{TRIGO}}(x) - f_{\text{Java}}(x)|}{|f_{\text{Java}}(x)|} \quad \text{si } f_{\text{Java}}(x) \neq 0$$

Dans la majorité des cas testés, l'erreur absolue est inférieure à quelques unités d'ULP, ce qui est cohérent avec les objectifs du projet.

11.6 Analyse des erreurs

L'analyse des erreurs montre plusieurs comportements intéressants :
— les erreurs sont minimales pour les valeurs proches de zéro,
— les zones proches des bornes du domaine (par exemple $x \approx \pm 1$ pour `asin`) sont plus sensibles,
— la réduction de domaine améliore fortement la convergence des séries de Taylor.

L'utilisation d'un critère d'arrêt basé sur l'ULP permet d'adapter dynamiquement le nombre de termes calculés, garantissant un compromis efficace entre précision et performance.

Globalement, les résultats obtenus confirment la validité de l'implémentation et montrent que l'extension TRIGO atteint un niveau de précision satisfaisant pour un langage pédagogique reposant uniquement sur des nombres flottants simple précision.

Chapitre 12

Conclusion

Ce projet avait pour objectif de concevoir et d'implémenter une extension TRIGO pour le langage Deca, en fournissant une bibliothèque mathématique complète et autonome, compatible avec les contraintes du langage et de sa machine abstraite. L'ensemble du travail a été mené en respectant une approche progressive, rigoureuse et orientée vers la compréhension fine des problématiques numériques.

12.1 Bilan

Le principal résultat de ce projet est la mise en œuvre fonctionnelle des principales fonctions trigonométriques et inverses :

$$\cos(x), \quad \sin(x), \quad \arcsin(x), \quad \arctan(x)$$

ainsi que de la fonction `ulp(float)`, indispensable à l'analyse de la précision numérique.

Ces fonctions ont été implémentées :

— sans recours à des bibliothèques externes,
— en simple précision (`float`),
— en utilisant exclusivement des méthodes mathématiques classiques (séries de Taylor, réductions de domaine, identités trigonométriques).

Les résultats obtenus montrent que l'extension TRIGO atteint une précision satisfaisante, généralement de l'ordre de quelques unités d'ULP, ce qui correspond aux objectifs réalistes fixés pour ce projet.

12.2 Apports techniques

Ce projet a permis d'aborder de nombreux aspects techniques avancés :

— la compréhension fine de la représentation des nombres flottants et de leurs limites,
— l'utilisation de séries de Taylor avec critères d'arrêt adaptatifs,
— la réduction de domaine pour améliorer la convergence numérique,
— l'évaluation de la précision via la notion d'ULP,
— l'analyse comparative avec une bibliothèque standard industrielle (Java `Math`).

Au-delà de l'implémentation elle-même, ce travail a également permis de développer une méthodologie rigoureuse de validation numérique, fondée sur des tests ciblés et une analyse critique des erreurs.

12.3 Difficultés rencontrées

Plusieurs difficultés ont été rencontrées tout au long du projet :

— la lente convergence de certaines séries de Taylor sur des domaines étendus,
— la gestion des cas limites, notamment pour les fonctions inverses,
— la propagation des erreurs d'arrondi en simple précision,
— le choix d'un compromis raisonnable entre précision et coût de calcul.

Ces difficultés ont nécessité des ajustements successifs, en particulier l'introduction systématique de réductions de domaine et de critères d'arrêt basés sur l'ULP, afin de garantir une stabilité numérique acceptable.

12.4 Conclusion générale

L'extension TRIGO développée dans ce projet constitue une implémentation cohérente, pédagogique et techniquement solide des fonctions trigonométriques en simple précision. Elle illustre concrètement les enjeux de l'implémentation numérique dans un langage de bas niveau, sans le support direct d'une bibliothèque mathématique avancée.

Ce travail met en évidence l'importance :
— d'une bonne compréhension mathématique des fonctions,
— d'une analyse fine des erreurs numériques,
— d'une approche progressive et méthodique du développement.

Enfin, ce projet s'inscrit pleinement dans les objectifs pédagogiques du cours de Génie Logiciel : concevoir un composant logiciel non trivial, en maîtrisant à la fois les aspects algorithmiques, numériques et ingénieriques.